Practical 8

Aim: Stock Market Analytics Using Window-Based Analysis.

Problem definition: You are a Data Analytics Engineer at FinEdge Securities Ltd., a financial services company that provides stock market insights to investors. The company continuously receives stock price updates from multiple exchanges. Investors want to understand stock performance trends and identify the best investment opportunities. Traditional data processing systems struggle to handle the continuously growing volume of financial data.

Observation / Output:

Task-1: Create Spark Session.

```
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 4.2.0
      /_/

Using Python version 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026 16:04:56)
Spark context Web UI available at http://Satyarajsinh:4040
Spark context available as 'sc' (master = local[*], app id = local-1790599551874).
SparkSession available as 'spark'.
>>>
```

Task-2: Load stock dataset into Spark DataFrame.

```
>>> from pyspark.sql.functions import *
>>> from pyspark.sql.window import Window
>>> df = spark.read.csv("C:/SEM5/BDA/Prac8/stock_data.csv", header=True, inferSchema=True)
```

Task-3: Display schema and stock records.

```
>>> df.printSchema()
root
 |-- Date: date (nullable = true)
 |-- Stock: string (nullable = true)
 |-- Price: double (nullable = true)
```

```
>>> df.show(10)
+----------+--------+------------------+
|      Date|   Stock|             Price|
+----------+--------+------------------+
|2025-09-29|RELIANCE| 1372.800048828125|
|2025-09-30|RELIANCE|            1364.0|
|2025-10-01|RELIANCE| 1368.699951171875|
|2025-10-03|RELIANCE|1363.4000244140625|
|2025-10-06|RELIANCE|            1375.0|
|2025-10-07|RELIANCE| 1384.800048828125|
|2025-10-08|RELIANCE|1367.4000244140625|
|2025-10-09|RELIANCE| 1377.800048828125|
|2025-10-10|RELIANCE| 1381.699951171875|
|2025-10-13|RELIANCE|            1375.0|
+----------+--------+------------------+
only showing top 10 rows
```

Task-4: Group records based on stock symbol.

```
>>> stock_groups = df.groupBy("Stock")
>>> df.select("Stock").distinct().show()
+--------+
|   Stock|
+--------+
|HDFCBANK|
|    INFY|
|RELIANCE|
|     TCS|
+--------+

>>>
```

Task-5: Calculate average stock price.

```
>>> average_price = df.groupBy("Stock").agg(
...     avg("Price").alias("Average_Price")
... )
>>> average_price.show()
+--------+------------------+
|   Stock|     Average_Price|
+--------+------------------+
|HDFCBANK| 851.6177290350317|
|    INFY|1315.4011972131007|
|RELIANCE|1388.2665368798243|
|     TCS|2628.8091583859873|
+--------+------------------+
```

Task-6: Calculate maximum stock price.

```
>>> maximum_price = df.groupBy("Stock").agg(
...     max("Price").alias("Maximum_Price")
... )
>>> maximum_price.orderBy("Stock").show(truncate=False)
+--------+-----------------+
|Stock   |Maximum_Price    |
+--------+-----------------+
|HDFCBANK|1009.5           |
|INFY    |1689.800048828125|
|RELIANCE|1592.300048828125|
|TCS     |3324.89990234375 |
+--------+-----------------+
```

Task-7: Calculate minimum stock price.

```
>>> minimum_price = df.groupBy("Stock").agg(
...     min("Price").alias("Minimum_Price")
... )
>>> minimum_price.orderBy("Stock").show(truncate=False)
+--------+------------------+
|Stock   |Minimum_Price     |
+--------+------------------+
|HDFCBANK|687.0999755859375 |
|INFY    |985.2999877929688 |
|RELIANCE|1197.5999755859375|
|TCS     |1982.5999755859375|
+--------+------------------+
```

Task-8: Generate stock summary report.

```
>>> summary = df.groupBy("Stock").agg(
...     avg("Price").alias("Average_Price"),
...     max("Price").alias("Maximum_Price"),
...     min("Price").alias("Minimum_Price")
... )
>>> summary.orderBy("Stock").show(truncate=False)
+--------+------------------+-----------------+------------------+
|Stock   |Average_Price     |Maximum_Price    |Minimum_Price     |
+--------+------------------+-----------------+------------------+
|HDFCBANK|851.6177290350317 |1009.5           |687.0999755859375 |
|INFY    |1315.4011972131007|1689.800048828125|985.2999877929688 |
|RELIANCE|1388.2665368798243|1592.300048828125|1197.5999755859375|
|TCS     |2628.8091583859873|3324.89990234375 |1982.5999755859375|
+--------+------------------+-----------------+------------------+

>>>
```

Task-9: Identify best-performing stocks.

```
>>> window = Window.partitionBy("Stock").orderBy("Date")
>>> df = df.withColumn(
...     "Previous_Price",
...     lag("Price").over(window)
... )
>>> df = df.withColumn(
...     "Daily_Return",
...     ((col("Price") - col("Previous_Price"))
...      / col("Previous_Price")) * 100
... )
```

```
>>> ascending_window = Window.partitionBy("Stock").orderBy("Date")
>>> start_price = df.withColumn(
...     "row_number",
...     row_number().over(ascending_window)
... ).filter(
...     col("row_number") == 1
... ).select(
...     "Stock",
...     col("Price").alias("Starting_Price")
... )
>>> descending_window = Window.partitionBy("Stock").orderBy(
...     col("Date").desc()
... )
>>> end_price = df.withColumn(
...     "row_number",
...     row_number().over(descending_window)
... ).filter(
...     col("row_number") == 1
... ).select(
...     "Stock",
...     col("Price").alias("Ending_Price")
... )
>>> performance = start_price.join(
...     end_price,
...     "Stock"
... )
>>> performance = performance.withColumn(
...     "Return_Percent",
...     ((col("Ending_Price") - col("Starting_Price"))
...      / col("Starting_Price")) * 100
... )
```

```
>>> performance.orderBy(
...     col("Return_Percent").desc()
... ).show(truncate=False)
+--------+----------------+----------------+-------------------+
|Stock   |Starting_Price  |Ending_Price    |Return_Percent     |
+--------+----------------+----------------+-------------------+
|RELIANCE|1372.800048828125|1197.5999755859375|-12.762242643547763|
|HDFCBANK|950.2999877929688|719.0499877929688 |-24.334421021836302|
|TCS     |2896.10009765625 |2070.699951171875 |-28.500401182692308|
|INFY    |1441.800048828125|1003.2000122070312|-30.420309458137538|
+--------+----------------+----------------+-------------------+

>>>
```

Task-10: Interpret market trends.

```
>>> moving_window = Window.partitionBy("Stock") \
...     .orderBy("Date") \
...     .rowsBetween(-2, 0)
>>> df = df.withColumn(
...     "Moving_Average_3D",
...     avg("Price").over(moving_window)
... )
>>> df.select(
...     "Date",
...     "Stock",
...     "Price",
...     "Moving_Average_3D"
... ).orderBy(
...     "Stock", "Date"
... ).show(20)
+----------+--------+------------------+------------------+
|      Date|   Stock|             Price|  Moving_Average_3D|
+----------+--------+------------------+------------------+
|2025-09-29|HDFCBANK| 950.2999877929688| 950.2999877929688|
|2025-09-30|HDFCBANK|             951.0| 950.6499938964844|
|2025-10-01|HDFCBANK|            965.25| 955.5166625976562|
|2025-10-03|HDFCBANK| 965.1500244140625| 960.4666748046875|
|2025-10-06|HDFCBANK| 973.4500122070312| 967.9500122070312|
|2025-10-07|HDFCBANK|             982.5| 973.7000122070312|
|2025-10-08|HDFCBANK| 978.7000122070312| 978.2166748046875|
|2025-10-09|HDFCBANK| 977.0999755859375| 979.4333292643229|
|2025-10-10|HDFCBANK| 980.9000244140625| 978.9000040690104|
|2025-10-13|HDFCBANK|             977.0| 978.3333333333334|
|2025-10-14|HDFCBANK| 977.1500244140625| 978.3500162760416|
|2025-10-15|HDFCBANK|            978.25| 977.4666748046875|
|2025-10-16|HDFCBANK| 994.3499755859375|            983.25|
|2025-10-17|HDFCBANK|1002.5499877929688| 991.7166544596354|
|2025-10-20|HDFCBANK|1002.9500122070312| 999.9499918619791|
|2025-10-21|HDFCBANK|1007.7000122070312|1004.4000040690104|
|2025-10-23|HDFCBANK|1008.7999877929688|1006.4833374023438|
|2025-10-24|HDFCBANK|            994.75|            1003.75|
|2025-10-27|HDFCBANK|1002.9500122070312|1002.1666666666666|
|2025-10-28|HDFCBANK|1003.5499877929688|1000.4166666666666|
+----------+--------+------------------+------------------+
only showing top 20 rows
```

Task-11: Final report

```
>>> volatility = df.groupBy("Stock").agg(
...     stddev("Daily_Return").alias("Volatility")
... )
>>> volatility.orderBy("Stock").show(truncate=False)
+--------+------------------+
|Stock   |Volatility        |
+--------+------------------+
|HDFCBANK|1.3618599351269376|
|INFY    |1.8888317512841706|
|RELIANCE|1.3056886078309398|
|TCS     |1.7731894828765997|
+--------+------------------+

>>> final_report = performance.join(
...     volatility,
...     "Stock"
... )
>>> final_report.orderBy(
...     col("Return_Percent").desc()
... ).show(truncate=False)
+--------+-----------------+------------------+-------------------+------------------+
|Stock   |Starting_Price   |Ending_Price      |Return_Percent     |Volatility        |
+--------+-----------------+------------------+-------------------+------------------+
|RELIANCE|1372.800048828125|1197.5999755859375|-12.762242643547763|1.3056886078309398|
|HDFCBANK|950.2999877929688|719.0499877929688 |-24.334421021836302|1.3618599351269376|
|TCS     |2896.10009765625 |2070.699951171875 |-28.500401182692308|1.7731894828765997|
|INFY    |1441.800048828125|1003.2000122070312|-30.420309458137538|1.8888317512841706|
+--------+-----------------+------------------+-------------------+------------------+
```

Task-12: Generate investment recommendations.

```
>>> recommendation = final_report.withColumn(
...     "Observation",
...     when(
...         (col("Return_Percent") > 0) &
...         (col("Volatility") < 3),
...         "Positive return with relatively lower volatility"
...     )
...     .when(
...         (col("Return_Percent") > 0) &
...         (col("Volatility") >= 3),
...         "Positive return with higher volatility"
...     )
...     .when(
...         col("Return_Percent") <= 0,
...         "Negative or flat return during the period"
...     )
...     .otherwise(
...         "Requires further analysis"
...     )
... )
>>> recommendation.orderBy(
...     col("Return_Percent").desc()
... ).show(truncate=False)
+--------+-----------------+------------------+-------------------+------------------+-----------------------------------------+
|Stock   |Starting_Price   |Ending_Price      |Return_Percent     |Volatility        |Observation                              |
+--------+-----------------+------------------+-------------------+------------------+-----------------------------------------+
|RELIANCE|1372.800048828125|1197.5999755859375|-12.762242643547763|1.3056886078309398|Negative or flat return during the period|
|HDFCBANK|950.2999877929688|719.0499877929688 |-24.334421021836302|1.3618599351269376|Negative or flat return during the period|
|TCS     |2896.10009765625 |2070.699951171875 |-28.500401182692308|1.7731894828765997|Negative or flat return during the period|
|INFY    |1441.800048828125|1003.2000122070312|-30.420309458137538|1.8888317512841706|Negative or flat return during the period|
+--------+-----------------+------------------+-------------------+------------------+-----------------------------------------+
```

Key Questions /Analysis:

Q1. Why is Apache Spark suitable for financial analytics?

Apache Spark is suitable for financial analytics because it can process large amounts of financial data quickly using distributed processing. It supports DataFrames, SQL, aggregation and window functions, which are useful for analysing stock prices, returns and trends.

Q2. What is the purpose of grouping stock data by stock symbol?

Grouping stock data by symbol separates the records of each company. It allows us to calculate individual statistics such as average, maximum, minimum price and returns for each stock.

Q3. How does average stock price help investors?

Average stock price gives an overall idea of the typical price level of a stock during a selected period. It can be used to compare price behaviour across different periods, but it should not be considered alone when making an investment decision.

Q4. Why are maximum and minimum prices important?

Maximum and minimum prices show the price range of a stock during the selected period. They help in understanding price fluctuations and identifying the highest and lowest levels reached by the stock.

Q5. How can Spark support real-time stock market monitoring?

Spark can support real-time stock market monitoring through Structured Streaming. It can continuously receive incoming market data, process it in near real time and calculate metrics such as price changes, returns and moving averages. This allows updated market information to be analysed as new data arrives.

Conclusion:

In this practical, stock market data was analysed using Apache Spark. The stock records were grouped according to their symbols, and average, maximum and minimum prices were calculated to create a summary of each stock. Window functions were also used to study price changes and moving averages. The analysis helped in understanding stock price behaviour and comparing historical performance. Overall, the practical demonstrated how Spark can be used for efficient financial data processing and can also be extended for real-time market analysis.